



**UNIVERSIDAD COMPLUTENSE
MADRID**

Actividad de Bases de datos SQL

Autor: Gerardo Fructuoso Vidal-Aragón

Profesor/a: Isabel Riomoros

Índice

1. Introducción	3
2. Diseño E-R.....	4
3. Diseño lógico	7
4. Implementación.....	10

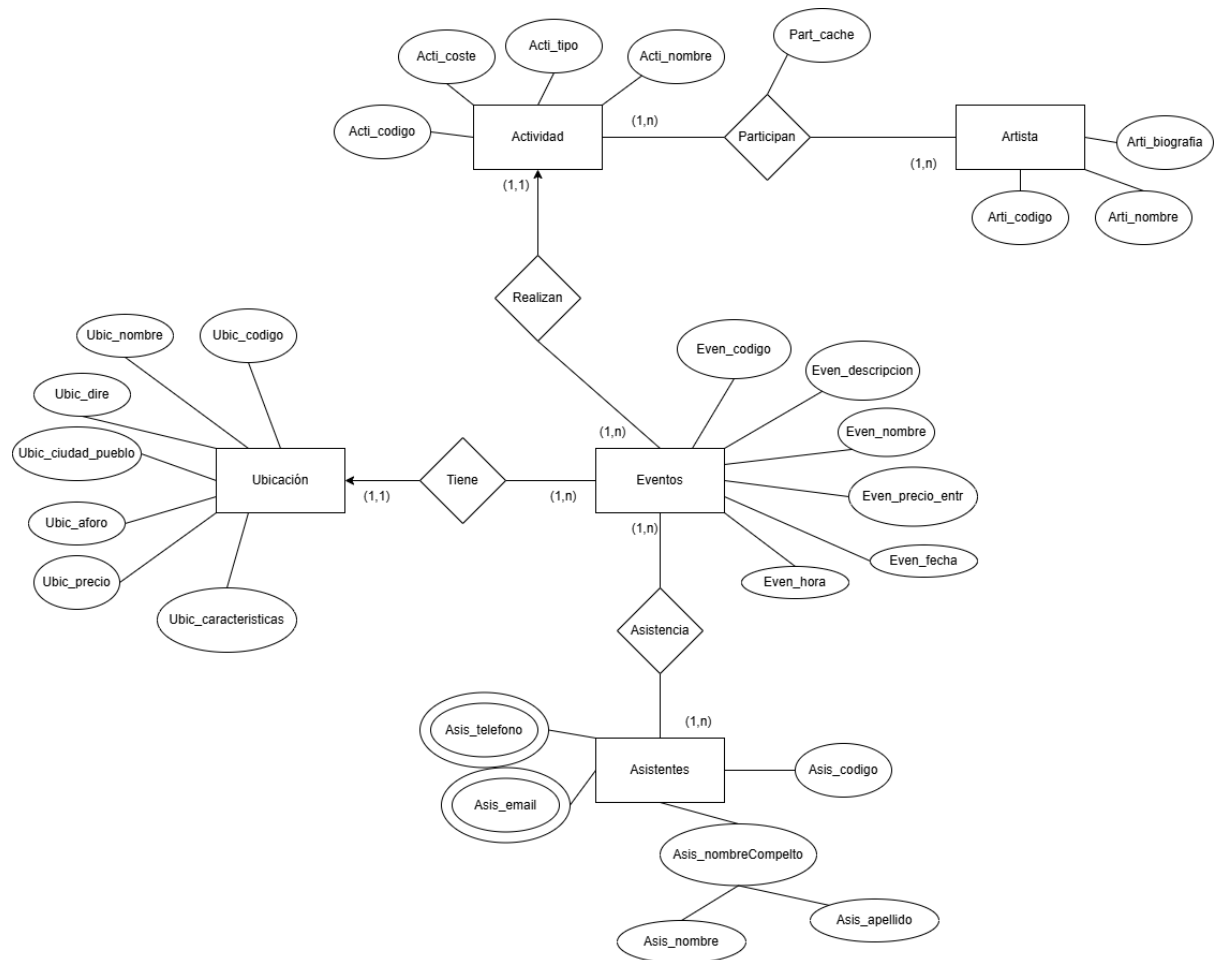
1. Introducción

En la asignatura de bases de datos SQL se ha aprendido desde realizar un diseño E-R hasta poder crear desde 0 una base de datos relacional. Esta actividad trata de crear desde 0 todo lo aprendido y se tiene que crear una base de datos (junto su diseño E-R) sobre una empresa conocida como “ArteVida Cultural” que se dedica a la organización de eventos culturales.

2. Diseño E-R

Para la creación del diseño E-R (Entidad - Relación) se ha utilizado como herramienta de creación del diagrama draw.io. Se ha hecho uso de esta herramienta ya que la había usado en anteriores ocasiones y es una herramienta muy útil y fácil de usar.

El diseño E-R que se ha creado es el siguiente:



Como se puede observar en este diagrama, se tienen 5 tablas principales las cuales están relacionadas entre sí. Evento tiene casi todas las relaciones, teniendo relación con: Actividad, Ubicación y Asistentes.

Luego está Artista que tiene relación con Actividad.

Estas tablas tienen los siguientes elementos:

- **Evento:**
 - o Even_codigo: Código único del evento
 - o Even_descripción: Descripción del evento
 - o Even_nombre: Nombre del evento

- Even_precio_entr: Precio de entrada del evento
- Even_fecha: Fecha del evento
- Even_hora: Hora del evento

Nota: Se ha separado la fecha y hora del evento para poder hacer queries más eficientes y simples como ejemplo: que eventos se realizarán en X fecha (así nos quitamos el tener que hacer uso de la hora y hacerla de forma más sencilla)

- **Asistentes:**

- Asis_codigo: Código del asistente
- Asis_telefono: Teléfonos del asistente
- Asis_email: Correos electrónicos del asistente
- Asis_nombreCompleto: Nombre completo del asistente que se divide en los siguientes campos:
 - Asis_nombre: Nombre del asistente
 - Asis_apellido: Apellido del asistente

- **Ubicación:**

- Ubic_codigo: Código de la ubicación
- Ubic_nombre: Nombre de la ubicación
- Ubic_dire: Dirección de la ubicación
- Ubic_ciudad_pueblo: Ciudad o pueblo de la ubicación
- Ubic_aforo: Aforo de la ubicación
- Ubic_precio: Precio de la ubicación
- Ubic_características: Características de la ubicación

- **Actividad:**

- Acti_codigo: Código de la actividad
- Acti_coste: Coste de la actividad
- Acti_tipo: Tipo de la actividad
- Acti_nombre: Nombre de la actividad

- **Artista:**

- Arti_biografia: Biografía del artista
- Arti_codigo: Código del artista
- Arti_nombre: Nombre del artista

Las relaciones creadas entre estas tablas son las siguientes:

- **Participan:** Esta es la relación que existe entre Actividad y Artista. Esta relación tiene una cardinalidad de N a N debido a que en una actividad pueden participar muchos artistas y un artista puede estar en muchas actividades. Esta relación tiene un campo que es **caché** y este campo se debe a que el caché de los artistas varía según la actividad en la que participen.
- **Realizan:** Esta es la relación que existe entre Evento y Actividad. Esta relación tiene una cardinalidad de 1 a N ya que en un evento solo puede haber una actividad mientras que una actividad puede estar en múltiples eventos.

- **Tiene:** Esta es la relación que existe entre Evento y Ubicación. Esta relación tiene una cardinalidad de 1 a N ya que un evento solo puede tener una ubicación mientras que una ubicación puede tener múltiples eventos.
- **Asistencia:** Esta es la relación que existe entre Evento y Asistentes. Esta relación tiene una cardinalidad de N a N ya que en un evento pueden asistir múltiples asistentes y un asistente puede participar en múltiples eventos.

Los distintos tipos de atributos que se han utilizado para la creación de este diagrama son los siguientes:

- **Simples:** La mayoría de los atributos son simples y estos son los que definen un campo de cada una de las tablas. Ejemplo: Ubic_codigo.
- **Compuesto:** Se ha hecho uso de un atributo compuesto y este se debe a que existía un nombre completo que poner el cual se divide en otros dos campos que son nombre y apellido. Ejemplo: Asis_nombreCompleto. Este se divide en: Asis_nombre y Asis_apellido
- **Multivalorado:** Se ha hecho uso de un par de atributos multivalorados y estos se deben a que puede llegar a tener más de un valor para un mismo código. Ejemplo: Asis_telefono. Un asistente puede llegar a tener más de un teléfono.

3. Diseño lógico

En este apartado se va a explicar y realizar el “paso a tablas” del diagrama realizado en el paso anterior explicando brevemente las acciones que se han tomado. También se indicará el tipo que se ha puesto, longitud y tipo de clave (si tiene).

Tablas creadas:

- **Evento:**
 - Even_codigo: Tipo INT y sería la Primary Key debido a que sería el código único.
 - Even_descripción: Varchar (50)
 - Even_nombre: Varchar(50)
 - Even_precio_entr: Double (se añaden decimales debido a que puede contenerlos)
 - Even_fecha: Date -> Solo fecha
 - Even_hora: Time -> Solo hora
 - Even_acti_codigo: Tipo INT y sería la foreign key debido a que ya existe como clave primaria en la tabla Actividades. Esto se debe a que tiene una relación de N a 1 donde la clave primaria del lado 1(Actividad) se pone en la del lado N(Evento)
 - Even_ubic_codigo: Tipo INT y sería la foreign key debido a que ya existe como clave primaria en la tabla Ubicaciones. Esto se debe a que tiene una relación de N a 1 donde la clave primaria del lado 1(Ubicaciones) se pone en la del lado N(Evento)
- **Asistentes:**
 - Asis_codigo: tipo INT y sería la Primary Key debido a que sería el código único.
 - Asis_nombre: Varchar(50)
 - Asis_apellido: Varchar(50)

Nota: Como los teléfonos de los asistentes y los emails son multivalorados, les creamos sus tablas ya que un mismo asis_codigo puede tener más de un teléfono o email.

- **Asis_telefono:**
 - Asis_código: Tipo INT y participa como Primary Key debido a que es el código del asistente. Además, este campo es también una foreign key ya que en la tabla de Asistentes está como PK.
 - Asis_prefijo: Varchar(4): Esto se debe a que pueden haber teléfonos internacionales. El 4 se debe a que el prefijo más grande son 3 dígitos y con el '+' son 4.

- Asis_telefono: Varchar(15): Esto se debe a que el teléfono más grande internacionalmente puede llegar a tener 15 dígitos. Este campo pertenece también en la primera Key.
- **Asis_email:**
 - Asis_código: Tipo INT y participa como Primary Key debido a que es el código del asistente. Además, este campo es también una foreign key ya que en la tabla de Asistentes está como PK.
 - Asis_email: Varchar (320): Esto se debe a que el email más grande que se puede crear son de 320 caracteres. Este campo pertenece también en la primera Key.
- **Ubicación:**
 - Ubic_codigo: Tipo INT y sería la Primary Key de esta tabla debido a que se trata de un código único
 - Ubic_nombre: Varchar(50)
 - Ubic_dire: Varchar(50)
 - Ubic_ciudad_pueblo: Varchar (50)
 - Ubic_aforo: INT
 - Ubic_precio: Double: Esto se debe a que el precio puede venir con decimales
 - Ubic_características: Varchar(200)
- **Actividad:**
 - Acti_codigo: Tipo INT y sería la Primary Key de esta tabla debido a que se trata de un código único
 - Acti_coste: Double: Se pone con decimales ya que el precio puede tener decimales
 - Acti_tipo: Varchar (50)
 - Acti_nombre: Varchar (100)
- **Artista:**
 - Arti_biografia: Biografía del artista
 - Arti_codigo: Código del artista
 - Arti_nombre: Nombre del artista

Nota: Estas tablas que se van a crear ahora son debido a la relación habida entre tablas de N a N.

- **Participan:**
 - Part_acti_codigo: Tipo INT y participa como Primary Key debido a que es el código de la actividad. Además, este campo es también una foreign key ya que en la tabla de Actividades está como PK.

- Part_arti_codigo: Tipo INT y participa como Primary Key debido a que es el código del artista. Además, este campo es también una foreign key ya que en la tabla de Artistas está como PK.
 - Caché: Double: Se debe a que el caché del artista puede tener decimales y puede ser distinto por cada actividad
- **Asistencia:**
- Asis_even_codigo: Tipo INT y participa como Primary Key debido a que es el código del evento. Además, este campo es también una foreign key ya que en la tabla de Eventos está como PK.
 - Asis_asis_codigo: Tipo INT y participa como Primary Key debido a que es el código del asistente. Además, este campo es también una foreign key ya que en la tabla de Asistentes está como PK.

4. Implementación

Una vez que tenemos el diagrama E-R y el diseño lógico, se debe proceder con la implementación de esta nueva base de datos. Para ello, se tiene que crear la base de datos y comenzar a usar dicha base de datos:

```
DROP DATABASE IF EXISTS ArteVidaCultural;
CREATE DATABASE ArteVidaCultural;
USE ArteVidaCultural;
```

Una vez se ha creado la base de datos, se tiene que proceder a la creación de tablas necesarias mencionadas en los apartados anteriores incluyendo todas las claves primarias y foráneas.

Nota: Se han puesto restricciones dentro de ciertos campos para que se cumpla con la condición indicada en dicho campo. Estas restricciones son:

- **NOT NULL:** Para que en dicho campo no se encuentren valores nulos, es decir, que sea un campo obligatorio.
- **UNIQUE:** Para aquellos cuyo valor del campo no puede estar repetido más veces en otros registros.
- **CHECK:** Para revisar que el dato metido en el campo del registro coincide con el formato que debería de tener.

Primero vamos a crear las tablas de Actividad y Ubicaciones ya que se usan sus claves principales como foráneas en la tabla evento.

Para la tabla Actividad:

```
CREATE TABLE ACTIVIDAD(  
ACTI_CODIGO INT AUTO_INCREMENT NOT NULL, -- LE PONEMOS QUE SE VAYA INCREMENTANDO  
SOLO TODOS LOS IDS DE LAS TABLAS  
ACTI_COSTE DOUBLE NOT NULL,  
ACTI_TIPO VARCHAR (50),  
ACTI_NOMBRE VARCHAR (100) NOT NULL,  
PRIMARY KEY(ACTI_CODIGO)  
);
```

Para la tabla de Ubicaciones:

```
CREATE TABLE UBICACIONES(  
UBIC_CODIGO INT AUTO_INCREMENT NOT NULL,  
UBIC_NOMBRE VARCHAR(50) NOT NULL,  
UBIC_DIRE VARCHAR(50) NOT NULL UNIQUE,  
UBIC_CIUDDAD_PUEBLO VARCHAR (50),  
UBIC_AFORO INT NOT NULL,  
UBIC_PRECIO DOUBLE NOT NULL,  
UBIC_CARACTERÍSTICAS VARCHAR(200),  
PRIMARY KEY(UBIC_CODIGO)  
);
```

Para la tabla Evento:

```
CREATE TABLE EVENTO(  
EVEN_CODIGO INT AUTO_INCREMENT NOT NULL,  
EVEN_DESCRIPCION VARCHAR(50),  
EVEN_NOMBRE VARCHAR(50) NOT NULL,  
EVEN_PRECIO_ENTR DOUBLE NOT NULL,  
EVEN_FECHA DATE NOT NULL,  
EVEN_HORA TIME NOT NULL,  
EVEN_ACTI_CODIGO INT,  
EVEN_UBIC_CODIGO INT,  
PRIMARY KEY(EVEN_CODIGO),  
FOREIGN KEY(EVEN_ACTI_CODIGO) REFERENCES ACTIVIDAD(ACTI_CODIGO),  
FOREIGN KEY(EVEN_UBIC_CODIGO) REFERENCES UBICACIONES(UBIC_CODIGO)  
);
```

Para la tabla Asistentes:

```
CREATE TABLE ASISTENTES(  
ASIS_CODIGO INT AUTO_INCREMENT NOT NULL,  
ASIS_NOMBRE VARCHAR(50) NOT NULL,  
ASIS_APELLIDO VARCHAR(50) NOT NULL,  
PRIMARY KEY (ASIS_CODIGO)  
);
```

Para la tabla de los teléfonos de los asistentes:

```
CREATE TABLE ASIS_TELEFONO(  

```

```
ASIS_CODIGO INT NOT NULL,  
ASIS_PREFIJO VARCHAR(4) NOT NULL,  
ASIS_TELEFONO VARCHAR(15) NOT NULL,  
PRIMARY KEY(ASIS_CODIGO, ASIS_PREFIJO, ASIS_TELEFONO),  
FOREIGN KEY (ASIS_CODIGO) REFERENCES ASISTENTES(ASIS_CODIGO)  
);
```

Para la tabla de los emails de los asistentes:

```
CREATE TABLE ASIS_EMAIL(  
ASIS_CODIGO INT NOT NULL,  
ASIS_EMAIL VARCHAR(320) NOT NULL,  
PRIMARY KEY(ASIS_CODIGO, ASIS_EMAIL),  
FOREIGN KEY (ASIS_CODIGO) REFERENCES ASISTENTES(ASIS_CODIGO)  
);
```

Para la tabla de los Artistas:

```
CREATE TABLE ARTISTA(  
ARTI_CODIGO INT AUTO_INCREMENT NOT NULL,  
ARTI_BIOGRAFIA VARCHAR(255),  
ARTI_NOMBRE VARCHAR(50) NOT NULL,  
PRIMARY KEY(ARTI_CODIGO)  
);
```

Para la tabla intermedia de Participan:

```
CREATE TABLE PARTICIPAN(  
PART_ACTI_CODIGO INT NOT NULL,  
PART_ARTI_CODIGO INT NOT NULL,  
LA_CACHE DOUBLE NOT NULL,  
PRIMARY KEY(PART_ACTI_CODIGO, PART_ARTI_CODIGO),  
FOREIGN KEY(PART_ACTI_CODIGO) REFERENCES ACTIVIDAD(ACTI_CODIGO),  
FOREIGN KEY(PART_ARTI_CODIGO) REFERENCES ARTISTA(ARTI_CODIGO)  
);
```

Para la tabla intermedia de Asistencia:

```
CREATE TABLE ASISTENCIA(  
ASIS_EVEN_CODIGO INT NOT NULL,  
ASIS_ASIS_CODIGO INT NOT NULL,  
PRIMARY KEY(ASIS_EVEN_CODIGO, ASIS_ASIS_CODIGO),  
FOREIGN KEY(ASIS_EVEN_CODIGO) REFERENCES EVENTO(EVEN_CODIGO),  
FOREIGN KEY(ASIS_ASIS_CODIGO) REFERENCES ASISTENTES(ASIS_CODIGO)  
);
```

Una vez se tienen creadas todas las tablas, procedemos a cargar estas tablas con los datos. Los datos se encuentran en la ruta siguiente:

```

105      -- Para saber en la ruta donde se tienen que agregar los archivos
106      SELECT @@GLOBAL.SECURE_FILE_PRIV;

```

Result Grid

	@@GLOBAL.SECURE_FILE_PRIV
▶	C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\

Una vez sabemos la ruta donde añadir los datos, procedemos a insertarlos.

Para la tabla de Actividades:

```

-- Una vez tenemos bien configurado todo, empezamos insertando por ejemplo los de
actividad
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Actividades.csv'
INTO TABLE ACTIVIDAD
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;

```

Comprobamos que se han insertado bien:

ACTI_CODIGO	ACTI_COSTE	ACTI_TIPO	ACTI_NOMBRE
1	10	Concierto	Concierto de musica clasica
2	20	Teatro	Teatro de ópera
3	30	Festival	Carnavales
4	40	Concierto	Concierto de musica clasica
5	50	Teatro	Teatro de ópera

Ahora seguimos con la tabla ubicaciones:

```

LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Ubicaciones.csv'
INTO TABLE UBICACIONES
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;

```

Comprobamos que se han cargado correctamente estos datos:

UBIC_CODIGO	UBIC_NOMBRE	UBIC_DIRE	UBIC_CIUADAD_PUEBLO	UBIC_AFORO	UBIC_PRECIO	UBIC_CARASTERÍSTICAS
1	Bernabeu	Paseo de la castellana	Madrid	5	20	Estadio de futbol para eventos
2	Wanda	Metropolitano	Madrid	3	10	Estadio de futbol para eventos
3	Nuevo vivero	Granadilla	Badajoz	2	3	Estadio de futbol para eventos
4	Torre eiffel	Centro	Paris	4	50	Monumento
5	NULL	NULL	NULL	NULL	NULL	NULL

Luego seguimos con la tabla de eventos:

```

LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Eventos.csv'
INTO TABLE EVENTO
FIELDS TERMINATED BY ','

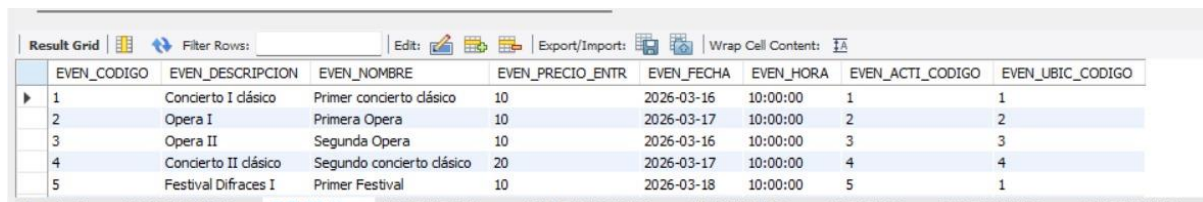
```

```

ENCLOSED BY '''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;

```

Comprobamos que se han cargado correctamente:



	EVEN_CODIGO	EVEN_DESCRIPCION	EVEN_NOMBRE	EVEN_PRECIO_ENTR	EVEN_FECHA	EVEN_HORA	EVEN_ACTI_CODIGO	EVEN_UBIC_CODIGO
1		Concierto I clásico	Primer concierto clásico	10	2026-03-16	10:00:00	1	1
2		Opera I	Primera Opera	10	2026-03-17	10:00:00	2	2
3		Opera II	Segunda Opera	10	2026-03-16	10:00:00	3	3
4		Concierto II clásico	Segundo concierto clásico	20	2026-03-17	10:00:00	4	4
5		Festival Difrances I	Primer Festival	10	2026-03-18	10:00:00	5	1

Luego seguimos con la tabla de Asistentes:

```

LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Asistentes.csv'
INTO TABLE ASISTENTES
FIELDS TERMINATED BY ','
ENCLOSED BY '''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;

```

Comprobamos que se han cargado correctamente:



	ASIS_CODIGO	ASIS_NOMBRE	ASIS_APELLIDO
1		JERRY	FRUCTUOSO
2		JERRY1	FRUCTUOSO1
3		JERRY2	FRUCTUOSO2
4		ISMAEL	ROMERO
5		ROSA	CASTELLANO

Ahora seguimos con la de los teléfonos de los asistentes:

```

LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Asis_telefono.csv'
INTO TABLE ASIS_TELEFONO
FIELDS TERMINATED BY ','
ENCLOSED BY '''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;

```

Comprobamos que se han cargado correctamente:

Result Grid			
		Filter Rows:	
Edit:			
	ASIS_CODIGO	ASIS_PREFIJO	ASIS_TELEFONO
▶	1	+34	324323422
	1	+34	678678678
	2	+361	423344224
	2	+361	698938472
	3	+34	323432233

Ahora seguimos con la tabla de los emails de los asistentes:

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Asis_email.csv'
INTO TABLE ASIS_EMAIL
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;
```

Comprobamos que se han cargado correctamente:

Result Grid		
		Filter Rows:
Edit:		
	ASIS_CODIGO	ASIS_EMAIL
▶	1	aaaaaa@aaaaaa.com
	1	eeeeeee@eeeeeee.com
	2	bbbbbb@bbbbbb.com
	2	rrrrrrrr@rrrrrrr.com
	3	ccccccc@cccccc.com

Ahora seguimos con la tabla de los artistas:

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Artista.csv'
INTO TABLE ARTISTA
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;
```

Comprobamos que se han cargado correctamente:

Result Grid			
		Filter Rows:	
Edit:			
	ARTI_CODIGO	ARTI_BIOGRAFIA	ARTI_NOMBRE
▶	1	Especializaod en musica clasica	Clasicazo
	2	Especializado en opera	Operazo
	3	Especializado en Festivales	Festivalazo
*	NULL	NULL	NULL

Seguimos con la tabla de los que participan (la conexion de artista y actividad):

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Participan.csv'
INTO TABLE PARTICIPAN
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;
```

Comprobamos que se han cargado correctamente:

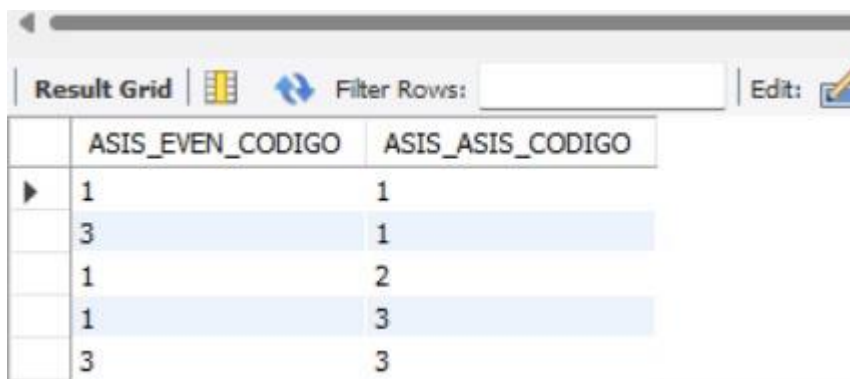


	PART_ACTI_CODIGO	PART_ARTI_CODIGO	LA_CACHE
▶	1	1	10
	2	2	20
	3	3	30
•	NULL	NULL	NULL

Por último, cargamos la tabla de Asistencia (la conexión entre envetos y asistentes):

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Asistencia.csv'
INTO TABLE ASISTENCIA
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;
```

Comprobamos que se han cargado correctamente:



	ASIS_EVEN_CODIGO	ASIS_ASIS_CODIGO
▶	1	1
	3	1
	1	2
	1	3
	3	3

Una vez que se ha terminado de insertar todos los datos necesarios para hacer las pruebas, procedemos a crear un trigger de ejemplo. Este trigger lo que va a hacer es verificar si ya hay un evento en la ubicación a la misma fecha y hora que otro (para que no se solape). Además de esto, comprueba 5 horas atrás y 5 horas después (más o menos se ha fijado 5 horas como duración de los eventos) para que no coincidan. El trigger es el siguiente:

```

DELIMITER //
DROP TRIGGER IF EXISTS COLAPSO_EVENTOS//
CREATE TRIGGER COLAPSO_EVENTOS
BEFORE INSERT ON EVENTO
FOR EACH ROW
BEGIN
IF EXISTS(
SELECT 1
FROM EVENTO
WHERE EVEN_UBIC_CODIGO = NEW.EVEN_UBIC_CODIGO
AND EVEN_FECHA = NEW.EVEN_FECHA
AND EVEN_HORA BETWEEN DATE_SUB(NEW.EVEN_HORA, INTERVAL 5 HOUR) AND
DATE_ADD(NEW.EVEN_HORA, INTERVAL 5 HOUR)
) THEN
    SIGNAL SQLSTATE '23000'
    SET MESSAGE_TEXT = " YA EXISTE UN EVENTO EN ESA UBI Y FECHA";
END IF;
END//
DELIMITER ;

```

Una vez creado, se comprueba que haya ido correctamente:

```

206 -- PROBAMOS QUE VA BIEN:
207 INSERT INTO EVENTO(EVEN_DESCRIPCION, EVEN_NOMBRE, EVEN_PRECIO_ENTR, EVEN_FECHA, EVEN_HORA, EVEN_ACTI_CODIGO, EVEN_UBIC_CODIGO)
208 VALUES ('Prueba no inserta','Prueba no inserta', 0,'2026-03-16','12:00:00',1,1);
209
210 INSERT INTO EVENTO(EVEN_DESCRIPCION, EVEN_NOMBRE, EVEN_PRECIO_ENTR, EVEN_FECHA, EVEN_HORA, EVEN_ACTI_CODIGO, EVEN_UBIC_CODIGO)
211 VALUES ('Prueba no inserta2','Prueba no inserta2', 0,'2026-03-16','10:00:00',1,1);
212

```

#	Time	Action	Message
65	17:27:03	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Artista.csv' INTO TABLE ARTIST...	3 row(s) affected Records: 3 Deleted: 0 Skipped: 0 Warnings: 0
66	17:27:03	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Participan.csv' INTO TABLE PAR...	3 row(s) affected Records: 3 Deleted: 0 Skipped: 0 Warnings: 0
67	17:27:03	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Asistencia.csv' INTO TABLE ASI...	9 row(s) affected Records: 9 Deleted: 0 Skipped: 0 Warnings: 0
68	17:27:03	SELECT * FROM ACTIVIDAD LIMIT 0, 1000	8 row(s) returned
69	17:27:03	SELECT * FROM UBICACIONES LIMIT 0, 1000	4 row(s) returned
70	17:27:03	SELECT * FROM EVENTO LIMIT 0, 1000	10 row(s) returned
71	17:27:03	SELECT * FROM ASISTENTES LIMIT 0, 1000	6 row(s) returned
72	17:27:03	SELECT * FROM ASIS_TELEFONO LIMIT 0, 1000	9 row(s) returned
73	17:27:03	SELECT * FROM ASIS_EMAIL LIMIT 0, 1000	9 row(s) returned
74	17:27:03	SELECT * FROM ARTISTA LIMIT 0, 1000	3 row(s) returned
75	17:27:03	SELECT * FROM PARTICIPAN LIMIT 0, 1000	3 row(s) returned
76	17:27:03	SELECT * FROM ASISTENCIA LIMIT 0, 1000	9 row(s) returned
77	17:27:03	DROP TRIGGER IF EXISTS COLAPSO_EVENTOS	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
78	17:27:03	CREATE TRIGGER COLAPSO_EVENTOS BEFORE INSERT ON EVENTO FOR EACH ROW BEGIN IF EXIS...	0 row(s) affected
79	17:27:03	-- PROBAMOS QUE VA BIEN: INSERT INTO EVENTO(EVEN_DESCRIPCION, EVEN_NOMBRE, EVEN_PR...	Error Code: 1644. YA EXISTE UN EVENTO EN ESA UBI Y FECHA

Como se puede observar en la imagen, ha fallado la inserción del evento por el error indicado por el trigger.

Una vez se ha creado un trigger, procedemos a realizar consultas. Se han realizado distintas consultas (de distintas dificultades) y son los siguientes:

- Consulta para ver los distintos tipos de actividad que hay:

```
SELECT DISTINCT(ACTI_TIPO) AS TIPO_DE_ACTIVIDAD FROM ACTIVIDAD;
```

TIPO_DE_ACTIVIDAD
Concierto
Teatro
Festival

- Para ver las distintas ubicaciones que hay:

```
SELECT UBIK_NOMBRE AS DISTINTAS_UBICACIONES, UBIK_DIRE AS DISTINTAS_DIRECCIONES,
UBIK_CIUADAD_PUEBLO AS DISTINTAS_CIUADADES
FROM UBICACIONES;
```

DISTINTAS_UBICACIONES	DISTINTAS_DIRECCIONES	DISTINTAS_CIUADADES
Bernabeu	Paseo de la castellana	Madrid
Wanda	Metropolitano	Madrid
Nuevo vivero	Granadilla	Badajoz
Torre eiffel	Centro	Paris

- Para ver los precios de las entradas de los eventos ordenados de menor a mayor:

```
SELECT EVEN_PRECIO_ENTR AS PRECIO_EVENTO, EVEN_NOMBRE AS NOMBRE_EVENTO FROM EVENTO
ORDER BY EVEN_PRECIO_ENTR ASC;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	PRECIO_EVENTO	NOMBRE_EVENTO			
▶	3	Cuarto concierto clásico			
	5	Segundo Festival			
	10	Primer concierto clásico			
	10	Primera Opera			
	10	Segunda Opera			
	10	Primer Festival			
	10	Tercer concierto clásico			
	20	Segundo concierto clásico			
	20	Tercer Festival			
	20	Tercer Festival			

- Para ver a que actividades participan que artistas y cuuya caché es mayor de 15:

```
SELECT ARTI_NOMBRE AS NOMBRE_DEL_ARTISTA, ACTI_NOMBRE AS NOMBRE_DE_ACTIVIDAD,
LA_CACHE AS CACHE_ARTISTA
FROM ARTISTA
INNER JOIN PARTICIPAN
ON ARTI_CODIGO = PART_ARTI_CODIGO
INNER JOIN ACTIVIDAD
ON ACTI_CODIGO = PART_ACTI_CODIGO
WHERE LA_CACHE > 15;
```

	NOMBRE_DEL_ARTISTA	NOMBRE_DE_ACTIVIDAD	CACHE_ARTISTA
▶	Operazo	Teatro de ópera	20
	Festivalazo	Carnavales	30

- Para ver la cantidad de dinero generada con las entradas ordenadas de mayor a menor:

```
SELECT EVEN_NOMBRE AS NOMBRE_EVENTO, EVEN_PRECIO_ENTR *
COALESCE(COUNT(DISTINCT(ASIS_ASIS_CODIGO)), 0) AS GANANCIAS
FROM EVENTO
LEFT JOIN ASISTENCIA
ON EVEN_CODIGO = ASIS_EVEN_CODIGO
GROUP BY EVEN_NOMBRE, EVEN_PRECIO_ENTR
ORDER BY GANANCIAS DESC;
```

Result Grid			Filter Rows:	Export:
	NOMBRE_EVENTO	GANANCIAS		
▶	Primer concierto clásico	30		
	Primera Opera	30		
	Segunda Opera	30		
	Cuarto concierto clásico	0		
	Primer Festival	0		
	Segundo concierto clásico	0		
	Segundo Festival	0		

- Se crea una vista con la que se pueda obtener el nombre, el telefono (el primero que se encuentre) y el email (el primero que se encuentre) de los asistentes:

```

DROP VIEW IF EXISTS VI_DATOS_ASISTENTES;
CREATE VIEW VI_DATOS_ASISTENTES AS
SELECT DISTINCT(A.ASIS_CODIGO) AS ASIS_CODIGO, A.ASIS_NOMBRE AS ASIS_NOMBRE,
MIN(CONCAT(T.ASIS_PREFIJO, ' ', T.ASIS_TELEFONO)) AS ASIS_TELEFONO,
MIN(E.ASIS_EMAIL) AS ASIS_EMAIL
FROM ASISTENTES A
LEFT JOIN ASIS_TELEFONO T
ON A.ASIS_CODIGO = T.ASIS_CODIGO
LEFT JOIN ASIS_EMAIL E
ON A.ASIS_CODIGO = E.ASIS_CODIGO
GROUP BY ASIS_CODIGO, ASIS_NOMBRE;

```

Comprobamos que ha ido correctamente la vista haciendole un select:

Result Grid					Filter Rows:	Export:	Wrap Cell Content:
	ASIS_CODIGO	ASIS_NOMBRE	ASIS_TELEFONO	ASIS_EMAIL			
▶	1	JERRY	+34 324323422	aaaaaa@aaaaaa.com			
	2	JERRY1	+361 423344224	bbbbbb@bbbbbb.com			
	3	JERRY2	+34 323432233	cccccc@cccccc.com			
	4	ISMAEL	+361 1233455234	dddddd@dddddd.com			
	5	ROSA	+34 678765354	@ .com			
	6	VINI	NULL	NULL			

- Para probar esta nueva vista vamos a hacer una query con ella. Se va a ver a que ubicaciones tienen que ir los asistentes incluyendo su nombre, telefono y email. También se verá el nombre del evento:

```

SELECT EVEN_NOMBRE AS NOMBRE_EVENTO, UBIC_NOMBRE AS NOMBRE_UBICACION, UBIC_DIRE AS
DIRECCION_UBICACION,
ASIS_NOMBRE AS NOMBRE_ASISTENTE, ASIS_TELEFONO AS TELEFONO_ASISTENTE, ASIS_EMAIL
AS EMAIL_ASISTENTE
FROM EVENTO
INNER JOIN UBICACIONES
ON UBIC_CODIGO = EVEN_UBIC_CODIGO
INNER JOIN ASISTENCIA
ON ASIS_EVEN_CODIGO = EVEN_CODIGO
INNER JOIN VI_DATOS_ASISTENTES
ON ASIS_CODIGO = ASIS_ASIS_CODIGO
ORDER BY NOMBRE_EVENTO;

```

	NOMBRE_EVENTO	NOMBRE_UBICACION	DIRECCION_UBICACION	NOMBRE_ASISTENTE	TELEFONO_ASISTENTE	EMAIL_ASISTENTE
►	Primer concierto clásico	Bernabeu	Paseo de la castellana	JERRY	+34 324323422	aaaaaa@aaaaaa.com
	Primer concierto clásico	Bernabeu	Paseo de la castellana	JERRY1	+361 423344224	bbbbbb@bbbbbb.com
	Primer concierto clásico	Bernabeu	Paseo de la castellana	JERRY2	+34 323432233	cccccc@cccccc.com
	Primera Opera	Wanda	Metropolitano	ISMAEL	+361 1233455234	dddddd@dddddd.com
	Primera Opera	Wanda	Metropolitano	ROSA	+34 678765354	@ .com
	Primera Opera	Wanda	Metropolitano	VINI	NULL	NULL
	Segunda Opera	Nuevo vivero	Granadilla	JERRY	+34 324323422	aaaaaa@aaaaaa.com
	Segunda Opera	Nuevo vivero	Granadilla	JERRY2	+34 323432233	cccccc@cccccc.com
	Segunda Opera	Nuevo vivero	Granadilla	VINI	NULL	NULL

- Ahora vamos a obtener la cantidad de los asistentes por cada uno de los eventos:

```

SELECT EVEN_NOMBRE AS NOMBRE_EVENTO, ARTI_NOMBRE AS NOMBRE_ARTISTA
FROM EVENTO
INNER JOIN ACTIVIDAD
ON ACTI_CODIGO = EVEN_ACTI_CODIGO
INNER JOIN PARTICIPAN
ON PART_ACTI_CODIGO = ACTI_CODIGO
INNER JOIN ARTISTA
ON PART_ARTI_CODIGO = ARTI_CODIGO
ORDER BY NOMBRE_ARTISTA;

```

	NOMBRE_EVENTO	CANTIDAD_ASISTENTES
►	Primer concierto clásico	3
	Primera Opera	3
	Segunda Opera	3
	Segundo concierto clásico	0
	Primer Festival	0
	Segundo Festival	0
	Tercer concierto clásico	0
	Tercer Festival	0
	Tercera Opera	0
	Cuarto concierto clásico	0

- Ahora vamos a ver que artistas asisten a que eventos:

```

SELECT EVEN_NOMBRE AS NOMBRE_EVENTO, ARTI_NOMBRE AS NOMBRE_ARTISTA
FROM EVENTO
INNER JOIN ACTIVIDAD
ON ACTI_CODIGO = EVEN_ACTI_CODIGO
INNER JOIN PARTICIPAN
ON PART_ACTI_CODIGO = ACTI_CODIGO
INNER JOIN ARTISTA
ON PART_ARTI_CODIGO = ARTI_CODIGO
ORDER BY NOMBRE_ARTISTA;

```

	NOMBRE_EVENTO	NOMBRE_ARTISTA
▶	Primer concierto clásico	Clasicazo
	Segunda Opera	Festivalazo
	Primera Opera	Operazo
	Tercera Opera	Operazo
	Cuarto concierto clásico	Operazo

- Se ha probado a realizar un insert normal de una ubicación nueva:

```

INSERT INTO UBICACIONES(UBIC_NOMBRE, UBIC_DIRE, UBIC_CIUADAD_PUEBLO,
UBIC_AFORO,UBIC_PRECIO)
VALUES("MONCLOA", "CALLE ISAAC PERAL", "MADRID", 100, 200);

```

Para ver si se ha insertado correctamente, vemos las ubicaciones con ese nombre:

```

SELECT * FROM UBICACIONES WHERE UBIC_NOMBRE = 'MONCLOA';

```

	UBIC_CODIGO	UBIC_NOMBRE	UBIC_DIRE	UBIC_CIUADAD_PUEBLO	UBIC_AFORO	UBIC_PRECIO	UBIC_CARACTERÍSTICAS
▶	5	MONCLOA	CALLE ISAAC PERAL	MADRID	100	200	NULL
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Ahora vamos a ver cuántos teléfonos tienen cada uno de los asistentes:

```

SELECT A.ASIS_NOMBRE AS NOMBRE_ASISTENTE, COALESCE(COUNT(B.ASIS_TELEFONO),0)
FROM ASISTENTES A
LEFT JOIN ASIS_TELEFONO B
ON A.ASIS_CODIGO = B.ASIS_CODIGO
GROUP BY A.ASIS_NOMBRE;

```

	NOMBRE_ASISTENTE	COALESCE(COUNT(B.ASIS_TELEFONO),0)
▶	JERRY	2
	JERRY1	2
	JERRY2	2
	ISMAEL	2
	ROSA	1
	VINI	0

- Hacemos lo mismo para ver la cantidad de correos de los asistentes:

```
SELECT A.ASIS_NOMBRE AS NOMBRE_ASISTENTE, COALESCE(COUNT(B.ASIS_EMAIL),0)
FROM ASISTENTES A
LEFT JOIN ASIS_EMAIL B
ON A.ASIS_CODIGO = B.ASIS_CODIGO
GROUP BY A.ASIS_NOMBRE;
```

	NOMBRE_ASISTENTE	COALESCE(COUNT(B.ASIS_EMAIL),0)
▶	JERRY	2
	JERRY1	2
	JERRY2	2
	ISMAEL	2
	ROSA	1
	VINI	0

- Obtenemos los eventos que hayan el 16 de marzo junto con sus horas:

```
SELECT EVEN_NOMBRE AS NOMBRE_EVENTO, EVEN_HORA AS HORA_EVENTO
FROM EVENTO
WHERE EVEN_FECHA = '2026-03-16';
```

	NOMBRE_EVENTO	HORA_EVENTO
▶	Primer concierto clásico	10:00:00
	Segunda Opera	10:00:00

- Por última consulta, vamos a ver cuantos eventos hay en las fechas que existan:

```
SELECT COUNT(DISTINCT(EVEN_CODIGO)) AS CANTIDAD_EVENTOS, EVEN_FECHA AS
FECHA_EVENTO
FROM EVENTO
GROUP BY EVEN_FECHA;
```

	CANTIDAD_EVENTOS	FECHA_EVENTO
▶	2	2026-03-16
	2	2026-03-17
	1	2026-03-18
	1	2026-03-19
	1	2026-03-20
	1	2026-03-21
	1	2026-03-22
	1	2026-03-23

5. Script

Se adjunta como texto el script copiado:

```
/----- Nombre del
autor: Gerardo Fructuoso Vidal-Aragón Nombre de la base de datos: ArteVidaCultural --
-----/* -----
----- Definición de la
estructura de la base de datos -----
-----// ----- INICIO CREACIÓN BASE DE DATOS -----
-----/ DROP DATABASE IF EXISTS ArteVidaCultural; CREATE DATABASE
ArteVidaCultural; USE ArteVidaCultural; / ----- FIN CREACIÓN BASE DE
DATOS -----//----- INICIO CREACIÓN DE TABLAS
-----*/ CREATE TABLE ACTIVIDAD( ACTI_CODIGO INT
AUTO_INCREMENT NOT NULL, -- LE PONEMOS QUE SE VAYA INCREMENTANDO SOLO
TODOS LOS IDS DE LAS TABLAS ACTI_COSTE DOUBLE NOT NULL, ACTI_TIPO
VARCHAR (50), ACTI_NOMBRE VARCHAR (100) NOT NULL, PRIMARY
KEY(ACTI_CODIGO) );

CREATE TABLE UBICACIONES( UBIC_CODIGO INT AUTO_INCREMENT NOT NULL,
UBIC_NOMBRE VARCHAR(50) NOT NULL, UBIC_DIRE VARCHAR(50) NOT NULL
UNIQUE, UBIC_CIUADAD_PUEBLO VARCHAR (50), UBIC_AFORO INT NOT NULL,
UBIC_PRECIO DOUBLE NOT NULL, UBIC_CARASTERÍSTICAS VARCHAR(200), PRIMARY
KEY(UBIC_CODIGO) );
```

```
CREATE TABLE EVENTO( EVEN_CODIGO INT AUTO_INCREMENT NOT NULL,
EVEN_DESCRIPCION VARCHAR(50), EVEN_NOMBRE VARCHAR(50) NOT NULL,
EVEN_PRECIO_ENTR DOUBLE NOT NULL, EVEN_FECHA DATE NOT NULL, EVEN_HORA
TIME NOT NULL, EVEN_ACTI_CODIGO INT, EVEN_UBIC_CODIGO INT, PRIMARY
KEY(EVEN_CODIGO), FOREIGN KEY(EVEN_ACTI_CODIGO) REFERENCES
ACTIVIDAD(ACTI_CODIGO), FOREIGN KEY(EVEN_UBIC_CODIGO) REFERENCES
UBICACIONES(UBIC_CODIGO) );
```

```
CREATE TABLE ASISTENTES( ASIS_CODIGO INT AUTO_INCREMENT NOT NULL,
ASIS_NOMBRE VARCHAR(50) NOT NULL, ASIS_APELLIDO VARCHAR(50) NOT NULL,
PRIMARY KEY (ASIS_CODIGO) );
```

```
CREATE TABLE ASIS_TELEFONO( ASIS_CODIGO INT NOT NULL, ASIS_PREFIJO
VARCHAR(4) NOT NULL, ASIS_TELEFONO VARCHAR(15) NOT NULL, CONSTRAINT
ASIS_PREFIJO CHECK(REGEXP_LIKE(ASIS_PREFIJO, '^\\+[0-9]{1,3}$')), CONSTRAINT
ASIS_TELEFONO CHECK (REGEXP_LIKE(ASIS_TELEFONO, '^[0-9]{8,15}$')), PRIMARY
KEY(ASIS_CODIGO, ASIS_PREFIJO, ASIS_TELEFONO), FOREIGN KEY (ASIS_CODIGO)
REFERENCES ASISTENTES(ASIS_CODIGO) );
```

```
CREATE TABLE ASIS_EMAIL( ASIS_CODIGO INT NOT NULL, ASIS_EMAIL VARCHAR(320)
NOT NULL, CONSTRAINT ASIS_EMAIL CHECK (REGEXP_LIKE(ASIS_EMAIL,
'^[\\^@]{1,64}@[\\^@]{1,255}$')), PRIMARY KEY(ASIS_CODIGO, ASIS_EMAIL), FOREIGN
KEY (ASIS_CODIGO) REFERENCES ASISTENTES(ASIS_CODIGO) );
```

```
CREATE TABLE ARTISTA( ARTI_CODIGO INT AUTO_INCREMENT NOT NULL,
ARTI_BIOGRAFIA VARCHAR(255), ARTI_NOMBRE VARCHAR(50) NOT NULL, PRIMARY
KEY(ARTI_CODIGO) );
```

```
CREATE TABLE PARTICIPAN( PART_ACTI_CODIGO INT NOT NULL, PART_ARTI_CODIGO
INT NOT NULL, LA_CACHE DOUBLE NOT NULL, PRIMARY KEY(PART_ACTI_CODIGO,
PART_ARTI_CODIGO), FOREIGN KEY(PART_ACTI_CODIGO) REFERENCES
ACTIVIDAD(ACTI_CODIGO), FOREIGN KEY(PART_ARTI_CODIGO) REFERENCES
ARTISTA(ARTI_CODIGO) );
```

```
CREATE TABLE ASISTENCIA( ASIS_EVEN_CODIGO INT NOT NULL, ASIS_ASIS_CODIGO
INT NOT NULL, PRIMARY KEY(ASIS_EVEN_CODIGO,ASIS_ASIS_CODIGO), FOREIGN
KEY(ASIS_EVEN_CODIGO) REFERENCES EVENTO(EVEN_CODIGO), FOREIGN
KEY(ASIS_ASIS_CODIGO) REFERENCES ASISTENTES(ASIS_CODIGO) ); /* -----
- FIN CREACIÓN DE TABLAS -----*/
```

```
/-----
Trigger Inserción de datos -----
-----/ /* Insertamos los datos desde los archivos que ya vienen
```

preparados. Estos archivos se encuentran dentro de la ruta: 'C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\' */ -- Para saber en la ruta donde se tienen que agregar los archivos SELECT @@GLOBAL.SECURE_FILE_PRIV; --

Una vez tenemos bien configurado todo, empezamos insertando por ejemplo los de actividad LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Actividades.csv' INTO TABLE ACTIVIDAD FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS; -- Ahora vamos con la de Ubicaciones LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Ubicaciones.csv' INTO TABLE UBICACIONES FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;

-- Ahora vamos con la de eventos LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Eventos.csv' INTO TABLE EVENTO FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS; -- Ahora vamos con la de Asistentes LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Asistentes.csv' INTO TABLE ASISTENTES FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;

-- Ahora vamos con la de Asis_telefono LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Asis_telefono.csv' INTO TABLE ASIS_TELEFONO FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS; -- Ahora vamos con la de Asis_email LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Asis_email.csv' INTO TABLE ASIS_EMAIL FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS; -- Ahora vamos con la de Artista LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Artista.csv' INTO TABLE ARTISTA FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS; -- Ahora vamos con la de Participan LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Participan.csv' INTO TABLE PARTICIPAN FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS; -- Ahora vamos con la de Asistencia LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Asistencia.csv' INTO TABLE ASISTENCIA FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;

-- PROBAMOS QUE LOS DATOS INSERTADOS SON CORRECTOS: SELECT * FROM ACTIVIDAD; SELECT * FROM UBICACIONES; SELECT * FROM EVENTO; SELECT * FROM ASISTENTES; SELECT * FROM ASIS_TELEFONO; SELECT * FROM ASIS_EMAIL; SELECT * FROM ARTISTA; SELECT * FROM PARTICIPAN; SELECT * FROM ASISTENCIA;

-- AHORA PROCEDEMOS A CREAR LOS TRIGGERS QUE SEAN NECESARIOS --
CREAMOS UN TRIGGER PARA VER SI A LA HORA DE CREAR UN EVENTO EN LA MISMA

```

UBICACION Y HORA HAY UNO YA CREADO: -- PONEMOS UN MINIMO DE QUE UN
EVENTO DURA 5 HORAS DELIMITER // DROP TRIGGER IF EXISTS COLAPSO_EVENTOS//
CREATE TRIGGER COLAPSO_EVENTOS BEFORE INSERT ON EVENTO FOR EACH ROW
BEGIN IF EXISTS( SELECT 1 FROM EVENTO WHERE EVEN_UBIC_CODIGO =
NEW.EVEN_UBIC_CODIGO AND EVEN_FECHA = NEW.EVEN_FECHA AND EVEN_HORA
BETWEEN DATE_SUB(NEW.EVEN_HORA, INTERVAL 5 HOUR) AND
DATE_ADD(NEW.EVEN_HORA, INTERVAL 5 HOUR) ) THEN SIGNAL SQLSTATE '23000'
SET MESSAGE_TEXT = " YA EXISTE UN EVENTO EN ESA UBI Y FECHA"; END IF; END//
DELIMITER ;

```

```

-- PROBAMOS QUE VA BIEN: /* INSERT INTO EVENTO(EVEN_DESCRIPCION,
EVEN_NOMBRE, EVEN_PRECIO_ENTR, EVEN_FECHA, EVEN_HORA,
EVEN_ACTI_CODIGO, EVEN_UBIC_CODIGO) VALUES ('Prueba no inserta','Prueba no
inserta', 0,'2026-03-16','12:00:00',1,1);

```

```

INSERT INTO EVENTO(EVEN_DESCRIPCION, EVEN_NOMBRE, EVEN_PRECIO_ENTR,
EVEN_FECHA, EVEN_HORA, EVEN_ACTI_CODIGO, EVEN_UBIC_CODIGO) VALUES
('Prueba no inserta2','Prueba no inserta2', 0,'2026-03-16','10:00:00',1,1); */ -- Funciona
correctamente, pero se comenta para que de fallo con el script ya que funciona
correctamente

```

```

/-----
Consultas, modificaciones, borrados y vistas con enunciado -----
-----/

```

```

/* Comenzamos con copnsultas sencillas a tablas simples */

```

```

-- Por ejemplo, vamos a ver los distintos tipos de actividades que hay: SELECT
DISTINCT(ACTI_TIPO) AS TIPO_DE_ACTIVIDAD FROM ACTIVIDAD;

```

```

-- Por ejemplo, vamos a ver las distintas ubicaciones que hay: SELECT UBIC_NOMBRE
AS DISTINTAS_UBICACIONES, UBIC_DIRE AS DISTINTAS_DIRECCIONES,
UBIC_CIUADAD_PUEBLO AS DISTINTAS_CIUADADES FROM UBICACIONES;

```

```

-- Por ejemplo ahora vamos a ver los precios de las entradas de los eventos ordenados
de menor a mayor (con su nombre) SELECT EVEN_PRECIO_ENTR AS PRECIO_EVENTO,
EVEN_NOMBRE AS NOMBRE_EVENTO FROM EVENTO ORDER BY EVEN_PRECIO_ENTR
ASC;

```

```

/* Comenzamos con consultas mas complejas (usando where y join) */ -- Por ejemplo,
vamos a ver a que actividades àrticipan que artistas cuyo caché es mayor a 15 SELECT
ARTI_NOMBRE AS NOMBRE_DEL_ARTISTA, ACTI_NOMBRE AS
NOMBRE_DE_ACTIVIDAD, LA_CACHE AS CACHE_ARTISTA FROM ARTISTA INNER JOIN

```

```
PARTICIPAN ON ARTI_CODIGO = PART_ARTI_CODIGO INNER JOIN ACTIVIDAD ON  
ACTI_CODIGO = PART_ACTI_CODIGO WHERE LA_CACHE > 15;
```

```
-- Por ejemplo vamos a ver la cantidad generada de dinero con las entradas. (Evento,  
asistencia) SELECT EVEN_NOMBRE AS NOMBRE_EVENTO, EVEN_PRECIO_ENTR *  
COALESCE(COUNT(DISTINCT(ASIS_ASIS_CODIGO)), 0) AS GANANCIAS FROM EVENTO  
LEFT JOIN ASISTENCIA ON EVEN_CODIGO = ASIS_EVEN_CODIGO GROUP BY  
EVEN_NOMBRE, EVEN_PRECIO_ENTR ORDER BY GANANCIAS DESC;
```

```
-- Por ejemplo, ahora vamos a crear una vista para obtener el nombre, telefono (el  
primero que encontremos) y email (primero que encontremos) de los asistentes DROP  
VIEW IF EXISTS VI_DATOS_ASISTENTES; CREATE VIEW VI_DATOS_ASISTENTES AS  
SELECT DISTINCT(A.ASIS_CODIGO) AS ASIS_CODIGO, A.ASIS_NOMBRE AS  
ASIS_NOMBRE, MIN(CONCAT(T.ASIS_PREFIJO, ' ', T.ASIS_TELEFONO)) AS  
ASIS_TELEFONO, MIN(E.ASIS_EMAIL) AS ASIS_EMAIL FROM ASISTENTES A LEFT JOIN  
ASIS_TELEFONO T ON A.ASIS_CODIGO = T.ASIS_CODIGO LEFT JOIN ASIS_EMAIL E ON  
A.ASIS_CODIGO = E.ASIS_CODIGO GROUP BY ASIS_CODIGO, ASIS_NOMBRE;
```

```
SELECT * FROM VI_DATOS_ASISTENTES;
```

```
-- Por ejemplo, vamos a ver ahora a que ubicaciones tienen que ir los asistentes según  
el evento al que tienen que ir, incluyendo su email y telefono -- Para esto, usamos la  
vista recientemente creada SELECT EVEN_NOMBRE AS NOMBRE_EVENTO,  
UBIC_NOMBRE AS NOMBRE_UBICACION, UBIC_DIRE AS DIRECCION_UBICACION,  
ASIS_NOMBRE AS NOMBRE_ASISTENTE, ASIS_TELEFONO AS TELEFONO_ASISTENTE,  
ASIS_EMAIL AS EMAIL_ASISTENTE FROM EVENTO INNER JOIN UBICACIONES ON  
UBIC_CODIGO = EVEN_UBIC_CODIGO INNER JOIN ASISTENCIA ON  
ASIS_EVEN_CODIGO = EVEN_CODIGO INNER JOIN VI_DATOS_ASISTENTES ON  
ASIS_CODIGO = ASIS_ASIS_CODIGO ORDER BY NOMBRE_EVENTO;
```

```
-- Por ejemplo vamos a ver la cantidad de asistentes por cada uno de los eventos  
SELECT EVEN_NOMBRE AS NOMBRE_EVENTO, COUNT(ASIS_ASIS_CODIGO) AS  
CANTIDAD_ASISTENTES FROM EVENTO LEFT JOIN ASISTENCIA ON EVEN_CODIGO =  
ASIS_EVEN_CODIGO GROUP BY EVEN_NOMBRE ORDER BY CANTIDAD_ASISTENTES  
DESC;
```

```
-- Por ejemplo, vamos a ver que artistas asisten a que eventos SELECT EVEN_NOMBRE  
AS NOMBRE_EVENTO, ARTI_NOMBRE AS NOMBRE_ARTISTA FROM EVENTO INNER  
JOIN ACTIVIDAD ON ACTI_CODIGO = EVEN_ACTI_CODIGO INNER JOIN PARTICIPAN  
ON PART_ACTI_CODIGO = ACTI_CODIGO INNER JOIN ARTISTA ON  
PART_ARTI_CODIGO = ARTI_CODIGO ORDER BY NOMBRE_ARTISTA;
```

```
-- VAMOS A HACER UN INSERT DE PRUEBA (NUEVA UBICACION) INSERT INTO
UBICACIONES(UBIC_NOMBRE, UBIC_DIRE, UBIC_CIUADAD_PUEBLO,
UBIC_AFORO,UBIC_PRECIO) VALUES("MONCLOA", "CALLE ISAAC PERAL", "MADRID",
100, 200);
```

```
-- comprobamos su correcta inserccion: SELECT * FROM UBICACIONES WHERE
UBIC_NOMBRE = 'MONCLOA';
```

```
-- POr ejemplo ahora vemos cuantos telefonos tienen cada uno de los asistentes
SELECT A.ASIS_NOMBRE AS NOMBRE_ASISTENTE,
COALESCE(COUNT(B.ASIS_TELEFONO),0) FROM ASISTENTES A LEFT JOIN
ASIS_TELEFONO B ON A.ASIS_CODIGO = B.ASIS_CODIGO GROUP BY
A.ASIS_NOMBRE;
```

```
-- POR EJEMPLO HACEMOS LO MISMO CON LOS CORREOS
```

```
SELECT A.ASIS_NOMBRE AS NOMBRE_ASISTENTE,
COALESCE(COUNT(B.ASIS_EMAIL),0) FROM ASISTENTES A LEFT JOIN ASIS_EMAIL B
ON A.ASIS_CODIGO = B.ASIS_CODIGO GROUP BY A.ASIS_NOMBRE;
```

```
-- Por ejemplo obeejetenmos los eventos que haya el 16 de marzo del 2026 Y SUS
HORAS
```

```
SELECT EVEN_NOMBRE AS NOMBRE_EVENTO, EVEN_HORA AS HORA_EVENTO FROM
EVENTO WHERE EVEN_FECHA = '2026-03-16'; -- POR EJEMPLO OBTENERMOS LAS
FECHAS CON LAS CANTIDADES DE EVENTOS QUE HAY EN DICHAS FECHAS SELECT
COUNT(DISTINCT(EVEN_CODIGO)) AS CANTIDAD_EVENTOS, EVEN_FECHA AS
FECHA_EVENTO FROM EVENTO GROUP BY EVEN_FECHA;
```